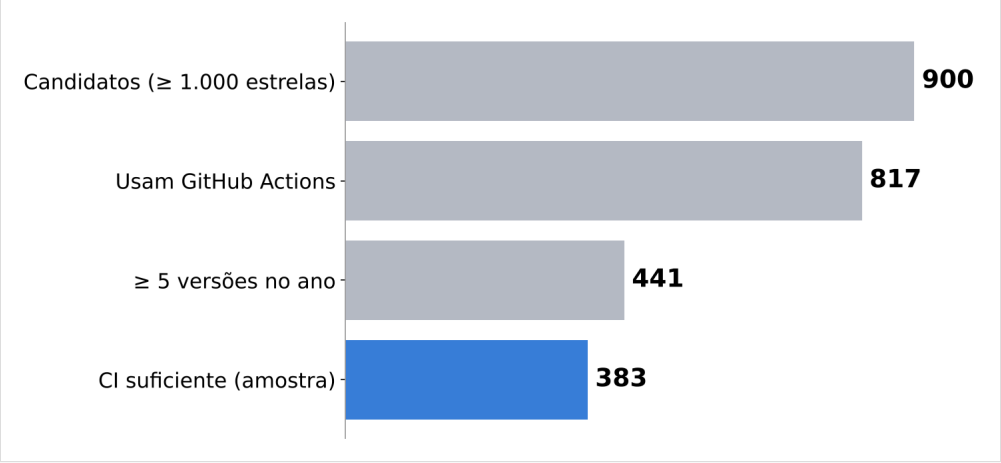
1 Metodologia: como coletamos

0:00 · 1 min



Artigo: Tabela 1 (funil de seleção), pág. 2

> "Começamos com 900 projetos populares e ficaram **383**, com um ano de dados cada (out/2025 a out/2026)."

O nosso diferencial: paralelizar ao máximo

- **Vários projetos ao mesmo tempo:** até 6 projetos coletados em paralelo, e dentro de cada um, 4 tarefas simultâneas (meses de execuções do CI e comparações entre versões).
- **Rodízio de chaves (tokens):** cada chave do GitHub tem um limite de consultas por hora. As threads dividem até 5 chaves, e cada consulta usa a chave que tem mais saldo. Quando uma acaba, as outras continuam.
- **Cache:** cada resposta fica salva. Se a coleta para no meio, ela continua de onde parou.

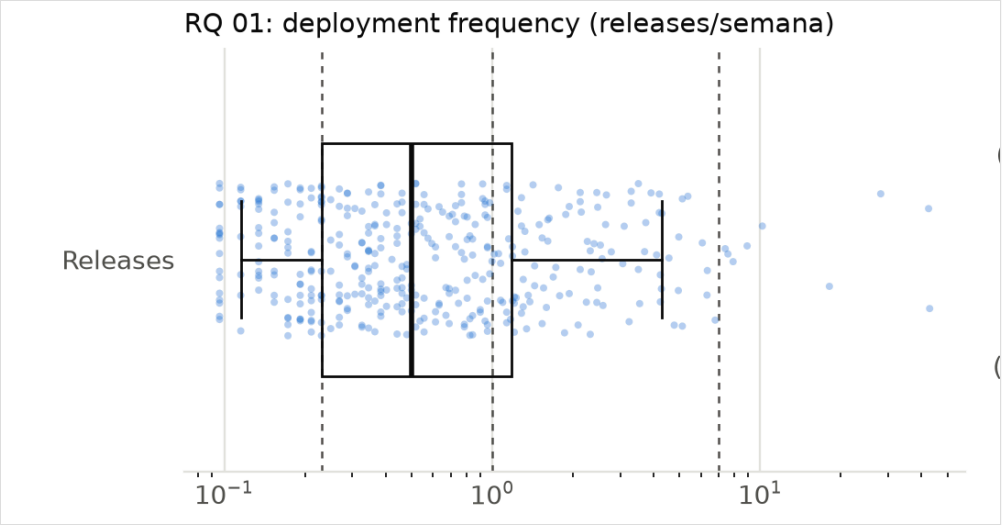
≈ 90 mil consultas à API do GitHub

> "Sem paralelizar, com uma chave só, a coleta levaria dias. Dividindo o trabalho entre threads e alternando as chaves, ela coube no prazo."

2 Resultados: velocidade (RQ 01 e RQ 02)

1:00

RQ 01: com que frequência publicam?



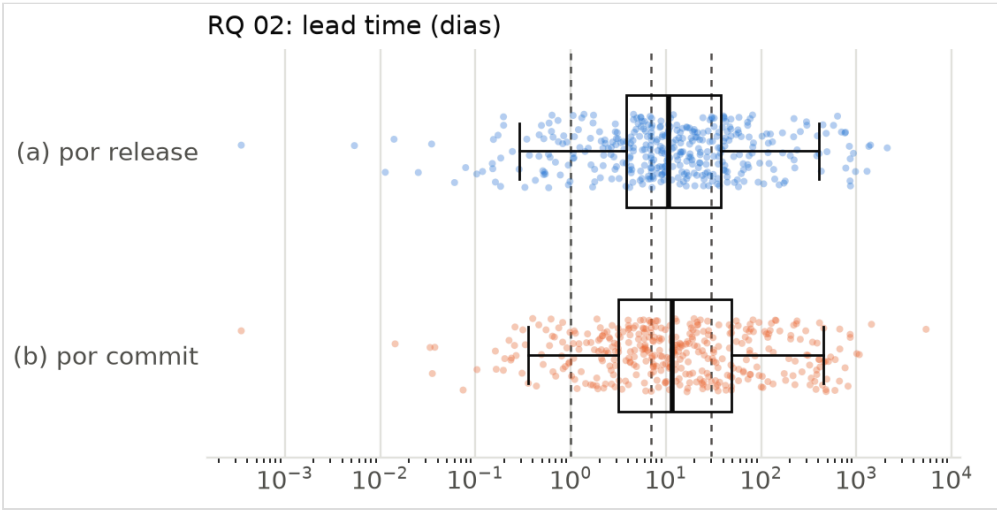
Artigo: Figura 1, painel RQ 01, pág. 5 · números na Tabela 4, pág. 4

1 versão a cada 2 semanas (0,50 por semana)

> Poucos fazem quase tudo: os 10% mais ativos (38 projetos) publicam 57% de todas as versões.

Esperávamos: muito desigual, concentrada em poucos → **confirmada**

RQ 02: quanto uma mudança demora para sair?



Artigo: Figura 1, painel RQ 02, pág. 5

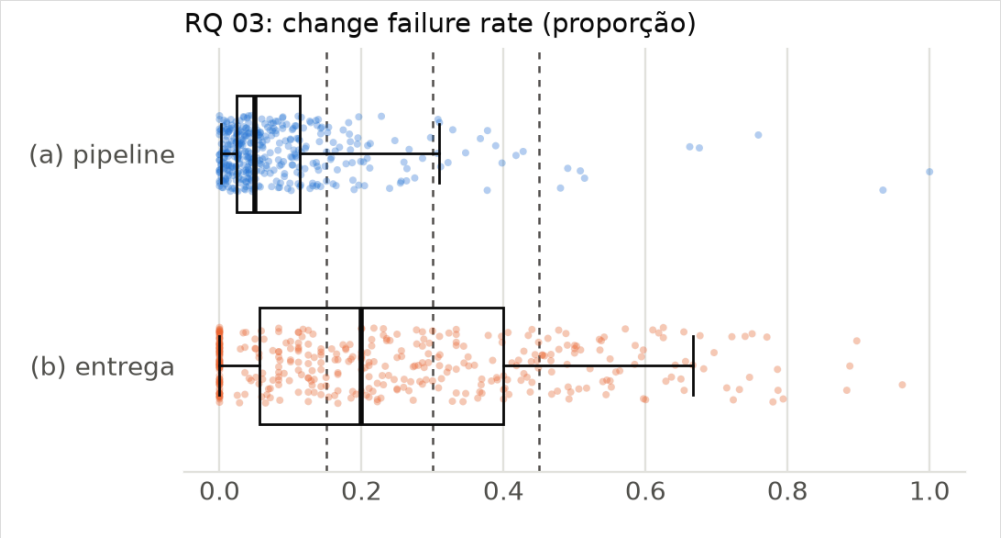
≈ 11 dias (10,7 por versão · 11,7 por commit)

> As duas formas de medir dão quase o mesmo valor típico.

Esperávamos: medir por commit daria menos → **parcial**, vale em 61% dos projetos

3 Resultados: estabilidade (RQ 03 e RQ 04)

RQ 03: quanto o projeto falha?



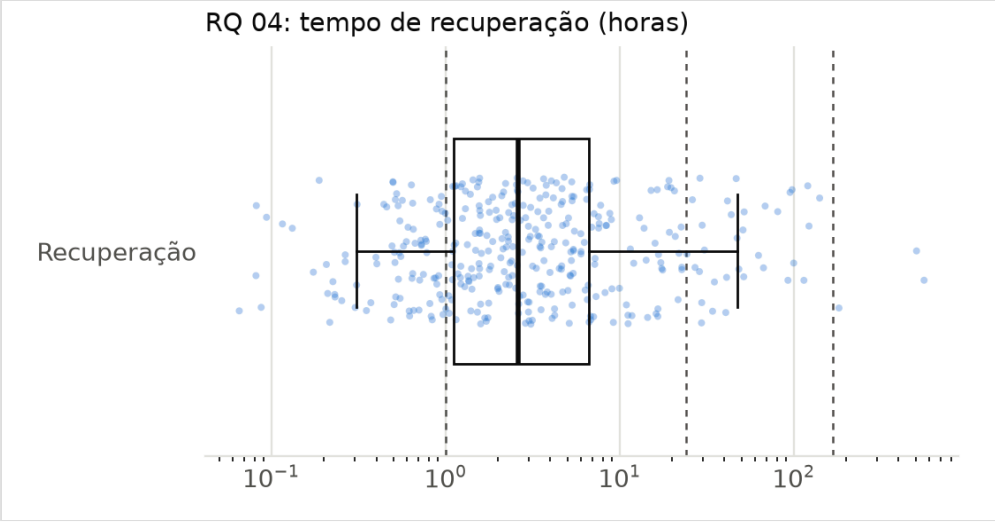
Artigo: Figura 1, painel RQ 03, pág. 5

Robô de testes: 5 em 100 · Versões: 20 em 100

> (a) de cada 100 execuções do robô de testes, 5 falham. (b) de cada 100 versões publicadas, 20 precisaram de conserto logo depois. Mesma pergunta, duas respostas.

Esperávamos: as duas formas não dariam o mesmo resultado → **confirmada**

RQ 04: quanto demora para consertar?



Artigo: Figura 1, painel RQ 04, pág. 5

2,6 horas para o robô voltar a passar

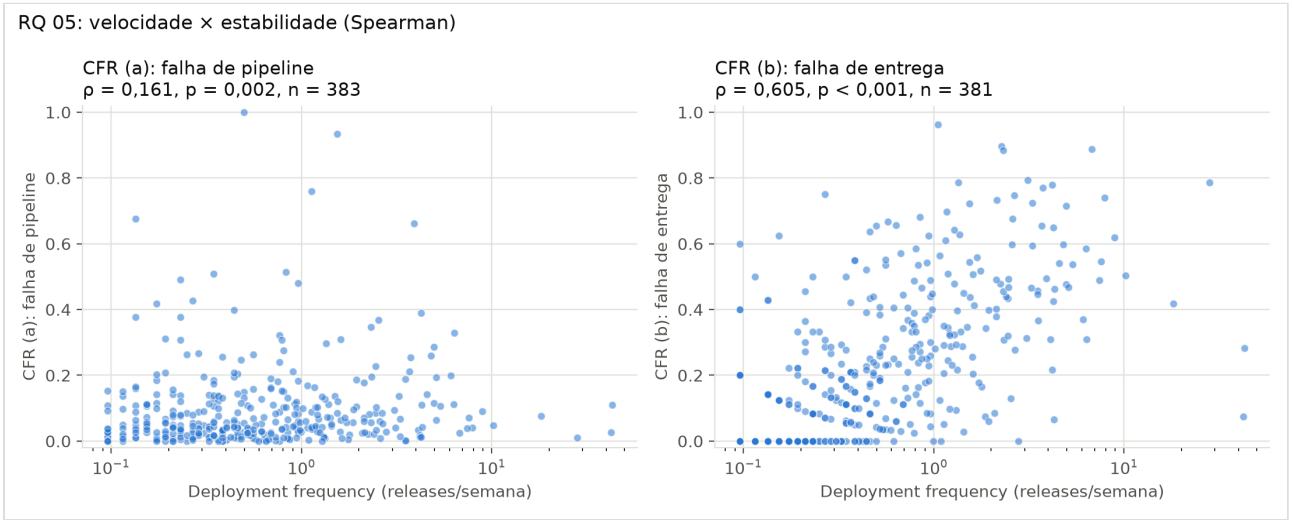
> Quando o robô de testes quebra, o típico é alguém consertar em menos de 3 horas. 90% dos projetos consertam em menos de 1 dia. Só 1% das quebras ficou sem conserto até o fim do ano.

Esperávamos: algumas falhas sem conserto no ano → **confirmada**, mas só 1%

4 Resultados: RQ 05 e RQ 08

2:00

RQ 05: quem entrega mais falha mais?



Artigo: Figura 2, pág. 5 · cada ponto é um projeto

Pelo robô: quase não (0,16) · Pelas versões: sim (0,61)

> À esquerda a nuvem é plana; à direita ela sobe. Parte da subida é efeito da regra: quem publica muito tem mais chance de ter duas versões em menos de 7 dias.

> Esperávamos: relação fraca ou nenhuma → **depende** da forma de medir falha

RQ 08 (bônus): quanto é retrabalho?

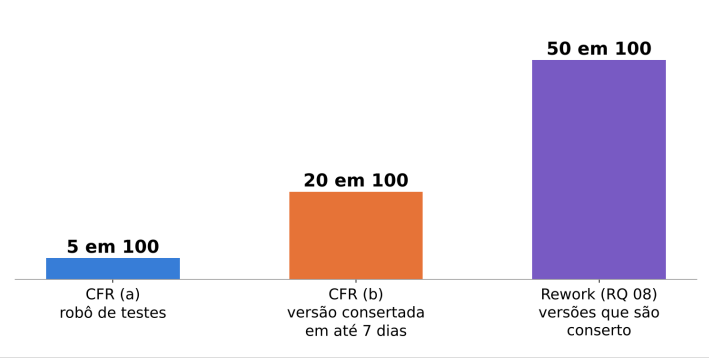


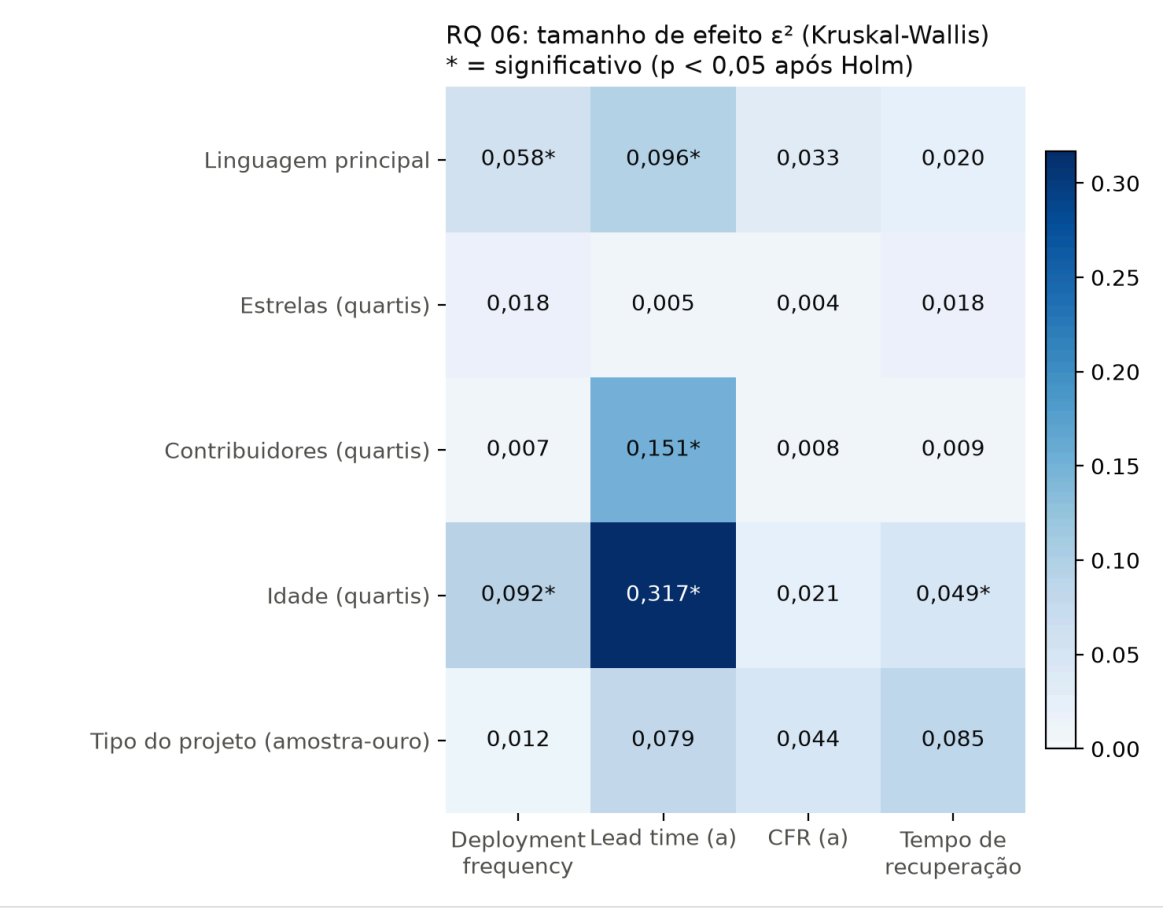
Gráfico feito com os números da Tabela 4 (pág. 4) e da RQ 08 (pág. 7)

De cada 100 versões, 50 são de conserto

> É mais que os 20 da taxa de falha porque aqui conta todo conserto; lá só conta a versão que quebrou e foi consertada em até 7 dias.

5 RQ 06: o que influencia o desempenho?

2:30 · 1 min



Artigo: Figura 3, pág. 6 · mais escuro = efeito maior · * = confiável depois da correção estatística

Idade pesa. Fama não.

- **Idade:** projetos novos levam **2,9 dias** para entregar uma mudança; os antigos, **42 dias**. É o quadrado mais escuro.
- **Estrelas:** a linha toda clara. Popularidade não muda nada.
- Contribuidores e linguagem também influenciam, mais fraco.
- Nenhuma característica explica a falha do robô de testes (coluna clara).

> "Fizemos 20 testes e usamos uma correção estatística para nenhum resultado aparecer por sorte."

Esperávamos: fama, idade, contribuidores e linguagem influenciariam → **parcial**: a fama não (a maior surpresa)

6 RQ 07: a classificação depende da forma de medir?

3:30

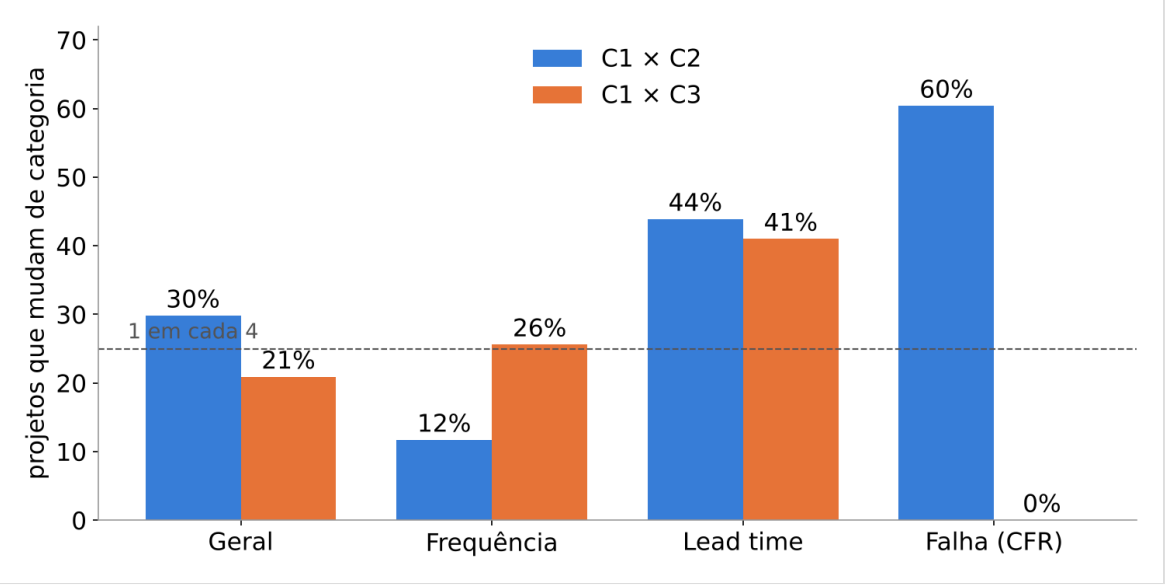


Gráfico feito com a Tabela 5, pág. 7 · C1 = referência · C2 e C3 = outras formas de medir

1 em cada 4 projetos muda de categoria

- Na classificação geral, de 21% a 30% mudam, quase sempre para a categoria vizinha (High ↔ Medium).
- **Na falha, 60% mudam.** Trocar "robô de testes" por "versões consertadas" praticamente embaralha quem é bom e quem é ruim (kappa 0,03, igual ao acaso).
- A frequência é a mais estável.

> Esperávamos: a categoria mudaria com a forma de medir → **confirmada**

7 Validação com humanos

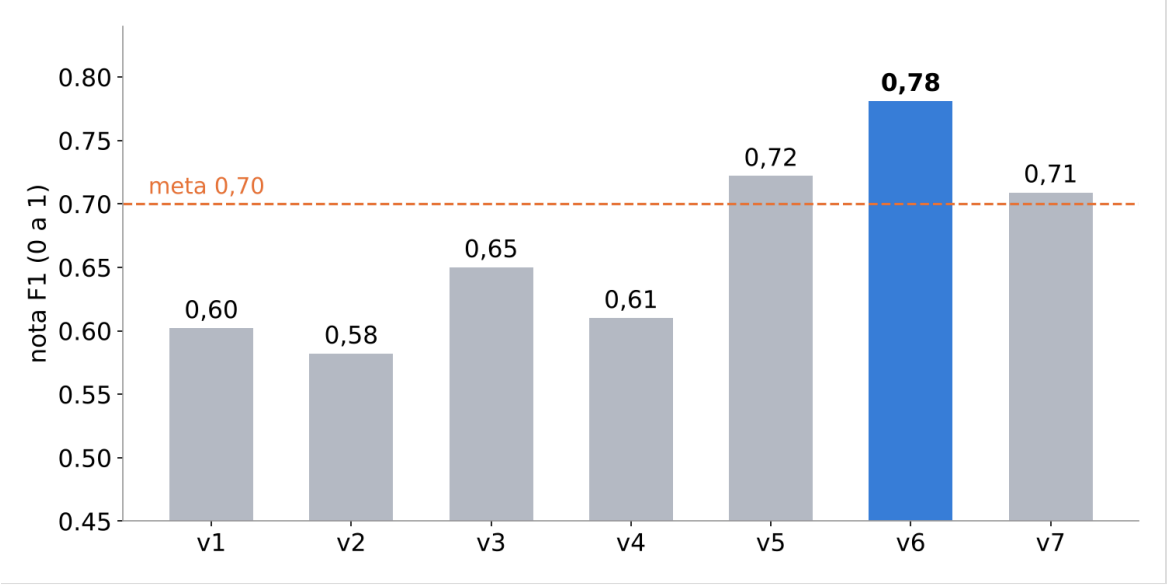


Gráfico feito com a Tabela 3, pág. 4 · concordância na Tabela 2, pág. 3

A regra automática passou da meta: 0,60 → 0,78

- Nós dois classificamos, cada um sozinho, **60 projetos e 300 versões**.
- Tipo do projeto: concordamos muito (kappa 0,88).
- "Essa versão é de conserto?": só moderado (0,48). Nem duas pessoas concordam bem.
- Os erros da regra eram versões de linhas antigas (ex.: PHP 8.4 publicada depois da 8.5). A v6 olha a própria versão e as notas de lançamento.

> Conferimos em 200 divisões aleatórias dos dados: a v6 fica acima da meta em 98% delas.

8 Conclusão

5:20 · 40 s

"A velocidade a gente mede de forma confiável; a estabilidade depende de como você define falha."

O que concluímos

- **Velocidade dá para medir, falha nem tanto.** O robô diz 5 em 100, as versões dizem 20. Mudando a forma de medir, 1 em cada 4 projetos muda de categoria; olhando só a falha, mais da metade.
- **A tese do DORA não se confirmou** com esses dados: quem entrega mais tende a falhar mais, dependendo de como se mede a falha.
- **Idade pesa, fama não.** Projetos novos entregam muito mais rápido; estrelas não mudam nada.
- **O projeto típico é mediano:** 43% High e 46% Medium; só 7% Elite.

Artigo: Seção 5 (Discussão) e Tabela 6 (hipóteses), pág. 7

O que o estudo contribui

- Um alerta: quem usa DORA em projetos abertos precisa **dizer como mediu a falha**, porque a conclusão muda.
- Mostramos que "versão de conserto" é difícil de definir até para pessoas (kappa 0,48).
- Melhoramos a regra automática de 0,60 para 0,78.
- Um pipeline que outro grupo roda com um comando, com coleta paralela e retomada pelo cache.